山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202500130002	姓名： 林毅玮	班级： 25.2
实验题目：实验 1		
实验学时：	实验日期：2026/9/16	
实验目的： 算法练习 1		
软件开发环境： VS code		

1. 实验内容

给定一个包含 n 个非负整数的序列 $a=[a_1,a_2,\ldots,a_n]$。定义子序列（要求保持原序列中元素的相对位置不变）的价值为 $\sum_{i=1}^{m} i\times a_i$（$m$ 为子序列长度，i 为元素在子序列中的位置）。要求编程求出所有子集（即子序列）价值的异或和。若为空序列，价值为 0

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

第一题

1 数据结构中使用动态数组来存储输入的队列，long long 来防止价值累加的时候发生整型溢出
递归栈：利用系统栈来实现深度优先搜索

2 算法对于序列中的每一个元素，都是有且仅有两种状态，选当前的子序列和不选入当前的子序列，可以通过深度优先搜索来遍历二叉树所有的情况，状态都有选当前的元素或者不选当前的元素，是否生成对应的子序列，达到边界条件 idx == n 的时候，说明生成对应的完整子序列，并且返回

3 复杂度分析：时间上复杂度：有二的 n 次方子序列，每个子序列在叶子结点上面进行异或操作
空间复杂度中：递归树的最大深度是 n，空间复杂度就是 O(n)

第二题

1 核心思想是利用 C++ 标准库<algorithm> 中的 std::next_permutation 函数，它能够按照字典序自动生成下一个排列。

2 前置条件是必须首先调用 sort(a.begin(), a.end())，将数组升序排序，以确保 next_permutation 能够从第一个排列开始生成。循环结构使用 do { ... } while (next_permutation(...))。在循环体内，计算当前排列的价值，并将其按位或到全局变量 ans 中

3 复杂度分析：
时间复杂度：全排列共有 $n!$ 种，计算每种排列的价值需要遍历 n 个元素，因此总时间复杂度为 $O(n\times n!)$空间复杂度：仅需存储数组和常数个变量，复杂度为 $O(n)$

3. 测试结果（测试输入，测试输出）

就是按照程序已给数据进行输入输出的测试

4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

第一题

异或运算（^）和加法运算（+）不满足分配律，即 (A + C) ^ (B + C) != (A ^ B) + C。因此，我们无法使用动态规划（DP）来合并状态，因为无法通过前 i 个元素的异或和推导出加入新元素后的异或和。同时，按位拆分也无法处理加法带来的进位问题。因此，对于 $n \leq 20$ 的数据范围，$O(2^n)$ 的 DFS 枚举是唯一的可行解法

若 $a_1 = 0$，由于 a_1 在子序列中的系数永远是 1，那么“选 a_1”和“不选 a_1”生成的子序列价值完全相同。这意味着所有子序列两两配对，异或结果为 0。可以在代码开头增加 if (a[0] == 0) { cout << 0; return; } 的 $O(1)$ 特判，能极大提高特定情况下的效率

问题二：

手写 DFS 生成全排列需要使用 used 数组，代码较长且对于有重复元素的序列（如 [2, 2, 2]）会生成重复的排列，造成计算浪费。而 std::next_permutation 不仅代码极其简洁，而且自带去重逻辑（只需先排序）

按位或运算具有“一旦为 1，永远为 1”的单调性。如果在计算过程中，ans 的所有有效二进制位都已经被置为 1（比如达到理论最大上限），理论上可以提前 break 退出循环。然而，由于每个排列的价值必须在计算完后才能知道，而计算价值需要 $O(n)$ 的时间，这种优化无法从根本上降低最坏情况下的时间复杂度，因此本题直接枚举即可。

虽然 $n \leq 10$，但 a_i 最大可达 100000，异或后加上位置系数，单个价值可能接近 10^6。虽然按位或不会导致数值疯狂增长（最多增长到接近数值的二进制全 1 状态），但在计算 current_val 时，养成使用 long long 的习惯可以有效避免潜在的溢出风险

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

```
//1. The value of subset
#include <iostream>
#include <vector>

using namespace std;

int n;
vector<long long> a;
long long ans = 0;

//idx : 当前遍历到的原数组下标
//len : 当前构建的子序列长度
//current_val : 当前子序列的积累得到的子集价值
```

```
void dfs(int idx, int len, long long current_val) {
    // 递归边界：已经遍历完了所有元素
    if (idx == n) {
        ans ^= current_val; //当前自序列的子集价值多加
        return;
    }

    // 选择 1：不选择当前的元素
    // 长度不变，价值不变，继续下一个元素
    dfs(idx + 1, len, current_val);

    // 选择 2：选入当前的元素，长度加一，子集价值增加
    // 长度加一，价值增加
    // len + 1  就是题目中的 i
    dfs(idx + 1, len + 1, current_val + (len + 1) * a[idx]);
}

int main() {
    cin >> n;
    a.resize(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    //从第零个元素开始：初始长度为 0，初始价值为 0
    dfs(0, 0, 0);

    cout << ans << endl;
    return 0;
}

//---
//2.求所有排列价值的按位或（STL 全排列）
// 核心是原数组要求原排列（求所有打乱顺序的可能性），计算价值，并且将价值进行按位或进行

#include <iostream>
#include <vector>
#include <algorithm>

// STL 中的 next_permutation 就是排列顺序的可能性

using namespace std;

int main() {
```

```
    int n;
    cin >> n;
    vector<int> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    // 首先需要排序，否则没法生成全部排列
    sort(a.begin(), a.end());

    long long ans = 0; //记录按位或的最终结果

    // do-while 循环进行按位或的结果
    do {
        long long current_val = 0;

        // 计算当前排列的价值
        for (int i = 0; i < n; ++i) {
            current_val += (a[i] ^ (i + 1));
        }

        // 将当前排列的价值按位或到总答案之中

        ans |= current_val;
    } while (next_permutation(a.begin(), a.end()));

    cout << ans << endl;
    return 0;
}
```